Risk-Aware Shortest-Path Search on a Dhaka OpenStreetMap Graph

Tanzila Khan

April 22, 2026

1 Introduction

Route planning in a megacity like Dhaka is not purely a distance-minimisation problem: travellers also try to avoid congested bus terminals, crowded marketplaces, and railway level crossings, and they prefer corridors that are near police stations, hospitals and fire stations. This combination of geometry and "soft" risk makes the routing task a natural test-bed for comparing uninformed and informed search algorithms under a *controllable* risk signal.

Algorithm selection

I picked the four classical uninformed and three informed algorithms that are both easy to implement and most illuminating under a "same setting, vary the algorithm" comparison:

- **Uninformed:** BFS, DFS, UCS, IDS.
- **Informed:** Greedy Best-First, A*, Weighted A* ($w = 1.5$).

I chose BFS/DFS as they provide a clean contrast between memory usage and completeness; UCS is the cost-optimal uninformed baseline; IDS combines BFS's optimality in hop count with DFS's memory profile. Greedy, A* and Weighted A* cover the spectrum of $f(n) = h(n)$, $f(n) = g(n) + h(n)$, and $f(n) = g(n) + w\,h(n)$ respectively. Running each informed algorithm under three different heuristics exposes the admissibility/speed trade-off within the same f-combination rule.

2 Dataset

The drivable road network of Dhaka city is loaded directly from OpenStreetMap by calling the following code snippets:

```
G = osmnx.graph_from_place("Dhaka, Bangladesh", network_type="drive",  simplify=True)
G = osmnx.distance.add_edge_lengths(G)
```

which returns a `networkx.MultiDiGraph` whose edges carry a `length` attribute in meters. For every amenity class used by the risk model we make a separate call

```
gdf = osmnx.features_from_place("Dhaka, Bangladesh", tags={tag_key: tag_value})
```

and extract each feature's centroid (ϕ, λ) in lat/long.

The table 1 reports the final sizes of the Dhaka's OSM. All six POI categories come from a single administrative boundary query (`Dhaka, Bangladesh`) so they are spatially consistent with the graph.

Table 1: Dhaka OSM dataset used in this run.

Entity	Count
Graph nodes	103,514
Graph edges	253,642
Average out-degree	≈ 2.45
Police stations	221
Hospitals	850
Fire stations	53
Bus stations	231
Marketplaces	985
Railway level crossings	894
Mean node risk $\bar{r}$	0.201

Note: Exact counts are reproducible given `osmnx.settings.use_cache=True` and the same OSM extract snapshot. The numbers in Table 1 reflect one such run at the time of writing.

3 Risk Model

I score each node with a factor-based formula. Every factor contributes a bounded additive term that decays linearly from its POI out to a radius R_k. The sign of the contribution is controlled by a polarity $\sigma_k \in \{+1, -1\}$.

3.1 Factors

Table 2: Factor definitions used in the Dhaka risk model.

Factor k	OSM tag	Polarity σ_k	Weight w_k	Radius R_k (m)
Police station	`amenity=police`	-1	0.30	500
Hospital	`amenity=hospital`	-1	0.25	400
Fire station	`amenity=fire_station`	-1	0.15	500
Bus terminal / stand	`amenity=bus_station`	$+1$	0.25	400
Marketplace	`amenity=marketplace`	$+1$	0.20	300
Level crossing	`railway=level_crossing`	$+1$	0.30	150
Random hazard zone	synthesised, 3% of nodes	$+1$	0.50	–

The **random hazard-zone** component models the "soft" hazards that OSM does not tag explicitly (mugging hotspots, monsoon flooding pockets, notorious pothole stretches). A reproducible random sample of 3% of the nodes is chosen with seed 42.

3.2 Per-factor contribution (kernel)

Let $d_k(n)$ denote the great-circle distance (in metres) from node n to the nearest instance of factor k. The contribution of factor k to node n is

$$c_k(n) \;=\; \sigma_k \, w_k \, \max\Big(0,\; 1 - \frac{d_k(n)}{R_k}\Big). \tag{1}$$

Three properties worth noting here:

1. At $d_k(n) = 0$ the contribution is $\sigma_k w_k$.
2. At $d_k(n) = R_k$ the contribution is zero.
3. Outside R_k the contribution is clamped to zero.

3.3 Node risk

The base risk accounts for the general urban-Dhaka background level; the time-of-day multiplier τ can be used to study night-time, rush-hour or monsoon scenarios without re-scoring. A hazard indicator $\mathbb{1}[n \in H]$ adds an extra bump for nodes that belong to the random hazard set H.

$$r(n) \;=\; \mathrm{clip}\Big(\big(r_{\text{base}} + \sum_k c_k(n) + \mathbb{1}[n \in H]\, r_{\text{haz}}\big) \cdot \tau \,,\; 0,\; 1\Big). \tag{2}$$

Default numerical values: $r_{\text{base}} = 0.20$, $r_{\text{haz}} = 0.50$, $|H| = \lceil 0.03\,|V| \rceil$, $\tau = 1.0$.

3.4 Edge risk

The edge-level risk is the simple average of the two endpoint risks,

$$r(u, v) \;=\; \tfrac{1}{2}\big(r(u) + r(v)\big), \tag{3}$$

which is what enters the cost simulator below.

3.5 Worked example

Let's consider a node n located 300 m from a police station, 100 m from a bus terminal, ≥ 400 m from the nearest hospital, and not in H. Plugging into (1) and (2) with $\tau = 1.0$:

$$\begin{aligned}
c_{\text{police}}(n) &= -0.30 \cdot \max(0,\, 1 - 300/500) = -0.120, \\
c_{\text{bus_station}}(n) &= +0.25 \cdot \max(0,\, 1 - 100/400) = +0.188, \\
c_{\text{hospital}}(n) &= 0, \\
r(n) &= \mathrm{clip}(0.20 - 0.120 + 0.188,\, 0,\, 1) = 0.268.
\end{aligned}$$

So this node's risk is roughly 27%, reflecting the intuition that it is mildly risky (close to a chaotic bus stand) but partially compensated by the nearby thana.

3.6 Vectorised implementation

For a city-scale graph ($|V| \approx 10^5$) the risk scoring is vectorised with NumPy using a flat-earth equirectangular approximation for the per-POI distance matrix. This is accurate within a few metres over urban distances and brings full-city scoring to under a second.

4 Cost Model

4.1 Search cost

Search uses the raw OSM edge length,

$$c_d(u, v) \;=\; \ell(u, v), \tag{4}$$

in metres, summed along the path:

$$G_d(\pi) \;=\; \sum_{(u,v)\in\pi} c_d(u, v). \tag{5}$$

For this, they all optimise (or try to) the same physical quantity.

4.2 Simulated actual cost

The *actual* cost, applied post-hoc to the chosen path, models what the user would experience if they really drove the route:

$$c_a(u, v) \;=\; \ell(u, v)\,\big(1 + \alpha\, r(u, v)\big)\,T\,\varepsilon, \tag{6}$$

with α a tunable risk weight, T a time-of-day multiplier, and $\varepsilon \sim \text{Uniform}(0.9,\, 1.15)$ a per-edge noise factor. The total simulated cost is $G_a(\pi) = \sum_{(u,v)\in\pi} c_a(u, v)$.

Separating G_d (used by the search) from G_a (used to judge a path *after* it is chosen) lets us study the effect of changing α without re-running any search — the path is fixed by G_d.

5 Heuristic Functions

All three heuristics return an estimate of the remaining distance (metres) from a node n to the fixed goal g. They differ in geometry and in whether they are admissible.

5.1 Euclidean (planar)

$$h_E(n) \;=\; \sqrt{\Delta x^2 + \Delta y^2}\,, \qquad \begin{aligned} \Delta y &= (\phi_g - \phi_n)\cdot 111\,320, \\ \Delta x &= (\lambda_g - \lambda_n)\cdot 111\,320\cdot\cos\bar{\phi}. \end{aligned} \tag{7}$$

Admissible: straight-line Euclidean distance is never larger than the road-network distance. Over a city-scale region the flat-earth approximation is tight to within metres.

5.2 Haversine (great circle)

$$h_H(n) \;=\; 2R\,\arcsin\sqrt{\sin^2\tfrac{\Delta\phi}{2} + \cos\phi_n\cos\phi_g\sin^2\tfrac{\Delta\lambda}{2}}, \tag{8}$$

with $R = 6{,}371{,}000\,\text{m}$. Admissible on the sphere, and geographically more faithful than h_E for larger regions.

5.3 Risk-weighted (inadmissible)

$$h_W(n) \;=\; h_E(n)\,\big(1 + w\,r(n)\big), \tag{9}$$

with w a weight (we use $w = 2.0$). By inflating h_E with the local risk score, h_W steers the search away from hazardous corridors. It is deliberately *not* admissible — it can exceed the true remaining c_d cost whenever $w\,r(n) > 0$ — so the resulting path is not guaranteed to be G_d-optimal. This is precisely what makes h_W interesting for comparison.

6 Search Algorithms

All seven algorithms share the same interface: a function that takes `(G, start, goal, ...)` and returns a `SearchResult` carrying the path, its g-cost, expansion / generation counts, peak frontier size, runtime, and depth.

6.1 Breadth-First Search (BFS)

BFS uses a FIFO queue with a *visited* set seeded at the start node: every node is generated at most once, so duplicate detection is aggressive. BFS returns the solution with the fewest *hops*, not the lowest distance cost. We still report the G_d of the BFS path for comparability with the optimisers.

6.2 Depth-First Search (DFS)

Iterative stack-based DFS with a global visited set. Because edge lengths are irrelevant to DFS, its first-found path typically zig-zags across the city for thousands of hops, which is exactly the point of the contrast with BFS / UCS.

6.3 Uniform-Cost Search (UCS)

Standard Dijkstra on the road graph: expand the frontier node with the smallest accumulated g. Equivalent to A* with $h \equiv 0$ and therefore G_d-optimal under non-negative edge weights.

6.4 Iterative Deepening Search (IDS)

A classical tree-search IDS: it runs a sequence of depth-limited DFS with limit $\ell = 0, 1, 2, \ldots$, reusing only path-ancestor cycle checking (not a cross-iteration visited set) so that completeness is preserved. A 30 s wall-clock cap is imposed as a safety net because, as the theory predicts, IDS is expected to time out on a graph with depth of the order of 400 hops and branching factor ≈ 2.5.

6.5 Greedy Best-First

Expand the frontier node with the smallest $h(n)$, regardless of g. We use each of the three heuristics h_E, h_H, h_W in turn.

6.6 A* Search

A* expands nodes in increasing order of

$$f(n) \;=\; g(n) + h(n), \tag{10}$$

where $g(n)$ is the best known c_d-cost from *start* to n and h is one of the three heuristics. We use the standard closed-set formulation with edge relaxation:

```
open  = min-heap seeded with (h(start), start)
g[start] = 0
while open non-empty:
    f, u = pop(open)
    if u in closed: continue
    if u == goal: return reconstruct(u)
    closed.add(u)
    for v in successors(u):
        tentative = g[u] + length(u, v)
        if tentative < g.get(v, inf):
            g[v] = tentative
            parent[v] = u
            push(open, (tentative + h(v), v))
```

The heap uses an integer tie-breaker so that nodes never have to be compared directly. We record the expansion order of every node for the heat-map visualisations.

6.7 Weighted A* (WA*)

Weighted A* modifies the evaluation to

$$f(n) \;=\; g(n) \;+\; w \cdot h(n), \qquad w \geq 1,$$

with $w = 1.5$ in our runs. At $w = 1$ it reduces to standard A*; at $w \to \infty$ it approaches Greedy. Even with an admissible h the path is no longer guaranteed to be optimal, but in exchange the search is much more goal-directed.

7 Metrics

7.1 Effective branching factor

The effective branching factor $b^\star$ is defined implicitly by

$$N + 1 \;=\; \sum_{i=0}^{d} (b^\star)^i \;=\; \frac{(b^\star)^{d+1} - 1}{b^\star - 1}, \tag{11}$$

with N the number of expansions and d the solution depth. We solve (11) by binary search over the interval $[1,\, N]$ to tolerance 10^{-4}. A $b^\star$ close to 1 indicates strong heuristic pruning: the search walked almost straight to the goal.

7.2 Full metric set

For every run we report:

- G_d (path g-cost, m) — (5).
- G_a (simulated actual cost, m) — (6).
- **Nodes expanded (N).**
- **Nodes generated.**

- **Peak frontier size.**
- **Wall-clock runtime.**
- **Depth** d (number of edges in the solution).
- **EBF** $b^\star$.

8 Experimental Setup

Graph. The whole city of Dhaka, Bangladesh, drive network from OSM, $|V| = 103{,}514$, $|E| = 253{,}642$.
Start / Goal. Selected reproducibly by seeded random sampling: both nodes are drawn uniformly from V, and the pair is retained only if the nominal shortest path between them spans at least 250 edges. With seed = 7 this yields `start=8222207234` and `goal=5626562516`.
Heuristics. h_E, h_H, h_W as defined in Section 5, with $w = 2.0$ for the risk-weighted heuristic.
Algorithm-specific settings. Weighted A* uses $w = 1.5$. IDS uses a 30 s wall-clock cap and a maximum depth of 500 to avoid running forever; it reports "NOT FOUND" if the cap is reached before the corresponding depth-limited DFS finds the goal.
Simulator settings. $T = 1.0$, $\varepsilon \sim \mathrm{U}(0.9, 1.15)$ (seed 0). Risk weight α is fixed at 1.5 for the main comparison, then swept over $\{0.5, 1.5, 3.0\}$ in a post-hoc re-simulation of the A*/h_E path (the search itself never sees α).
Hardware / software. macOS (Darwin 25.4.0), Python 3.13, `osmnx`, `networkx`, `matplotlib`, `numpy`. Full environment pinned in `pyproject.toml`.

9 Results

9.1 A* with three heuristics

Table 3 shows the effect of changing only the heuristic while holding A* fixed. Figures **??** through 3 show the corresponding Dhaka maps with the exploration heat-map in viridis (yellow = early expansion, purple = late), the solution path in red, and safety/hazard POIs overlaid as blue '+' / red 'x' markers.

Table 3: A* on the full Dhaka graph, same start/goal, three heuristics.

Heuristic	Depth	G_d (m)	G_a (m)	Expanded	Generated	Max front.	Runtime (s)	$b^\star$
Euclidean	399	173,686	235,615	67,049	77,135	1,300	0.231	1.018
Haversine	399	173,686	235,615	67,236	77,364	1,305	0.233	1.018
Risk-weighted	490	190,383	255,562	50,969	59,853	3,541	0.163	1.013

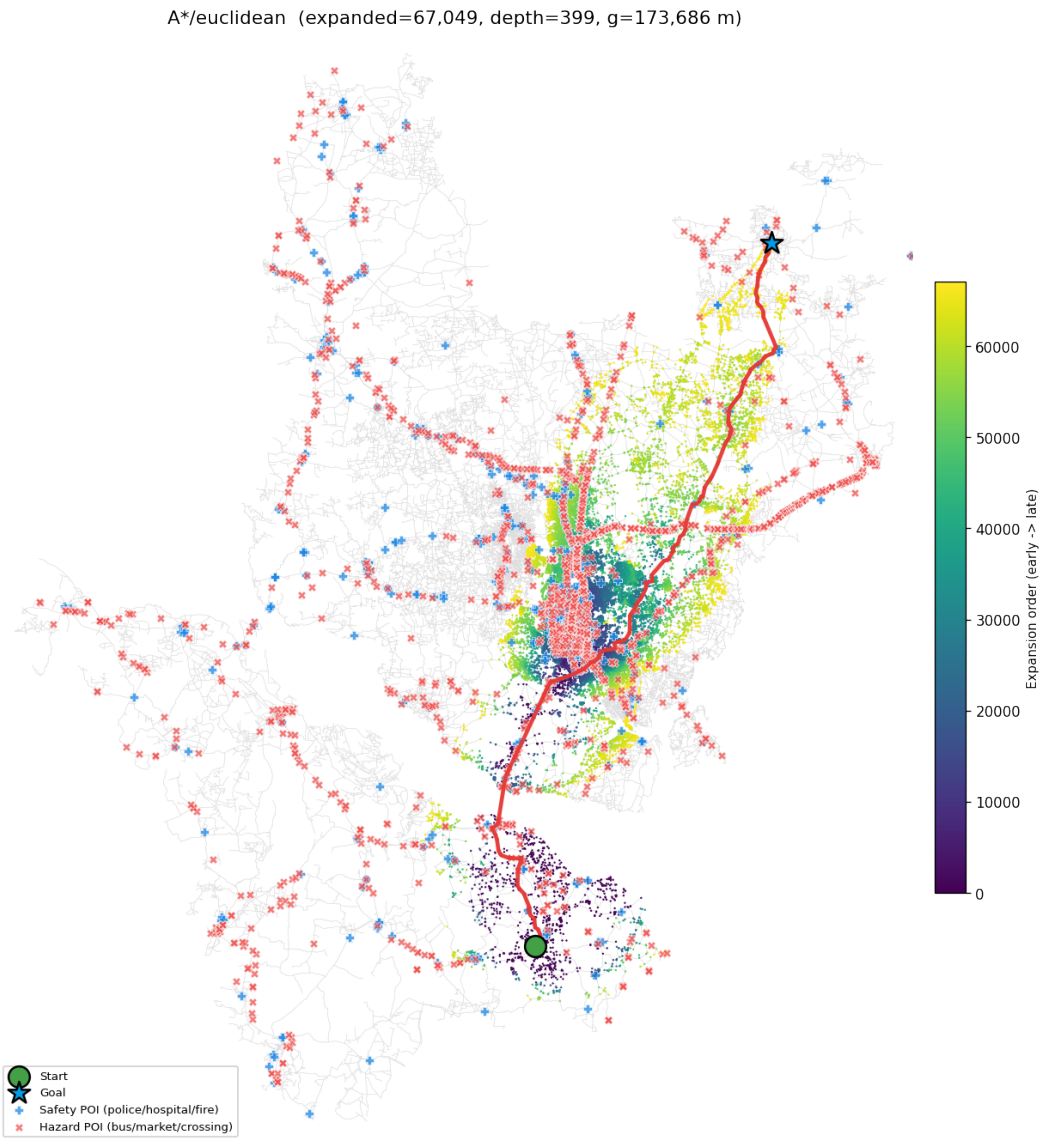


Figure 1: A* with Euclidean heuristic. Viridis = expansion order, red line = found path, green circle = start, cyan star = goal, blue '+' = safety POI, red 'x' = hazard POI.

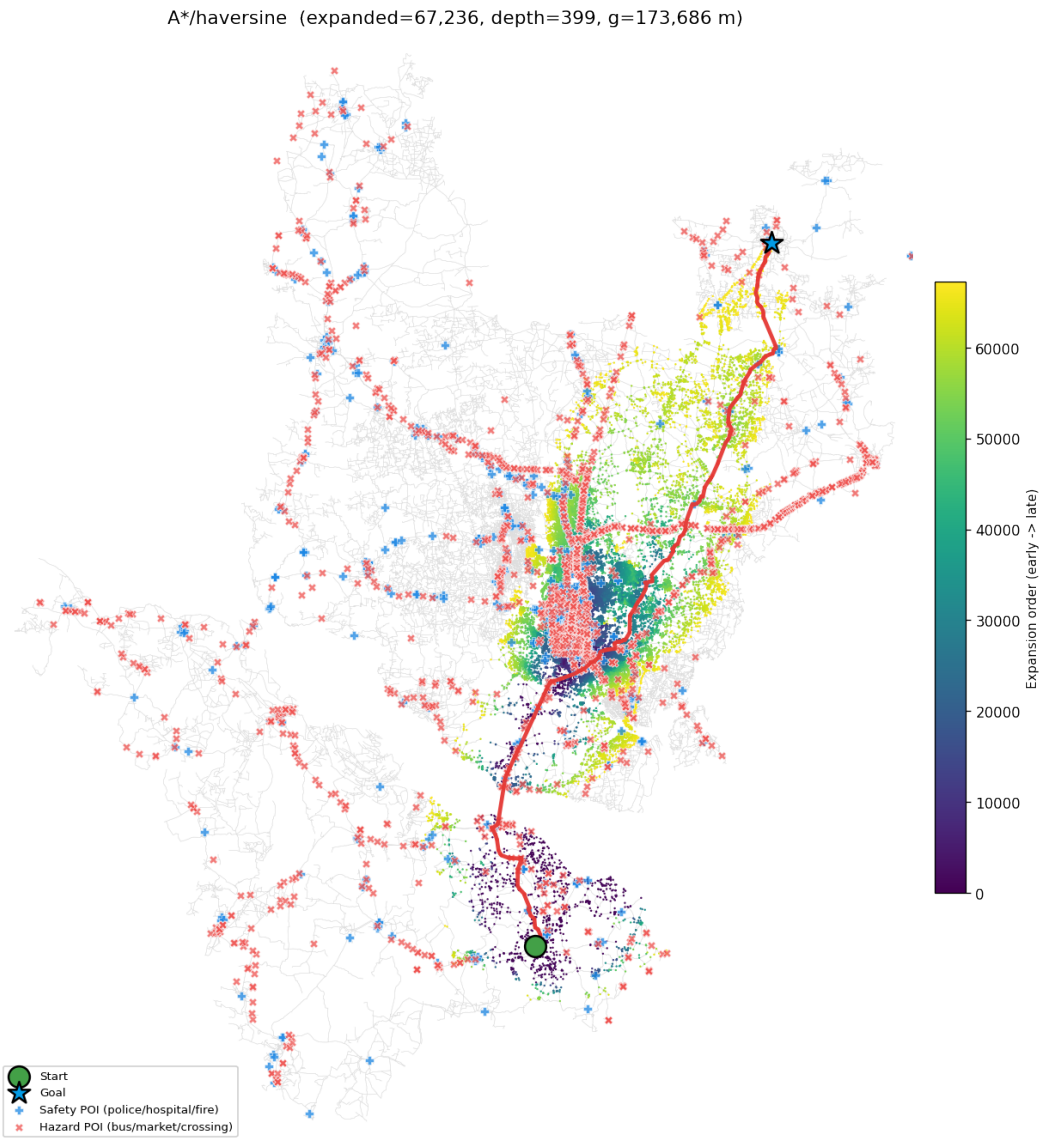


Figure 2: A* with the Haversine heuristic. The exploration pattern is visually indistinguishable from the Euclidean run at Dhaka scale.

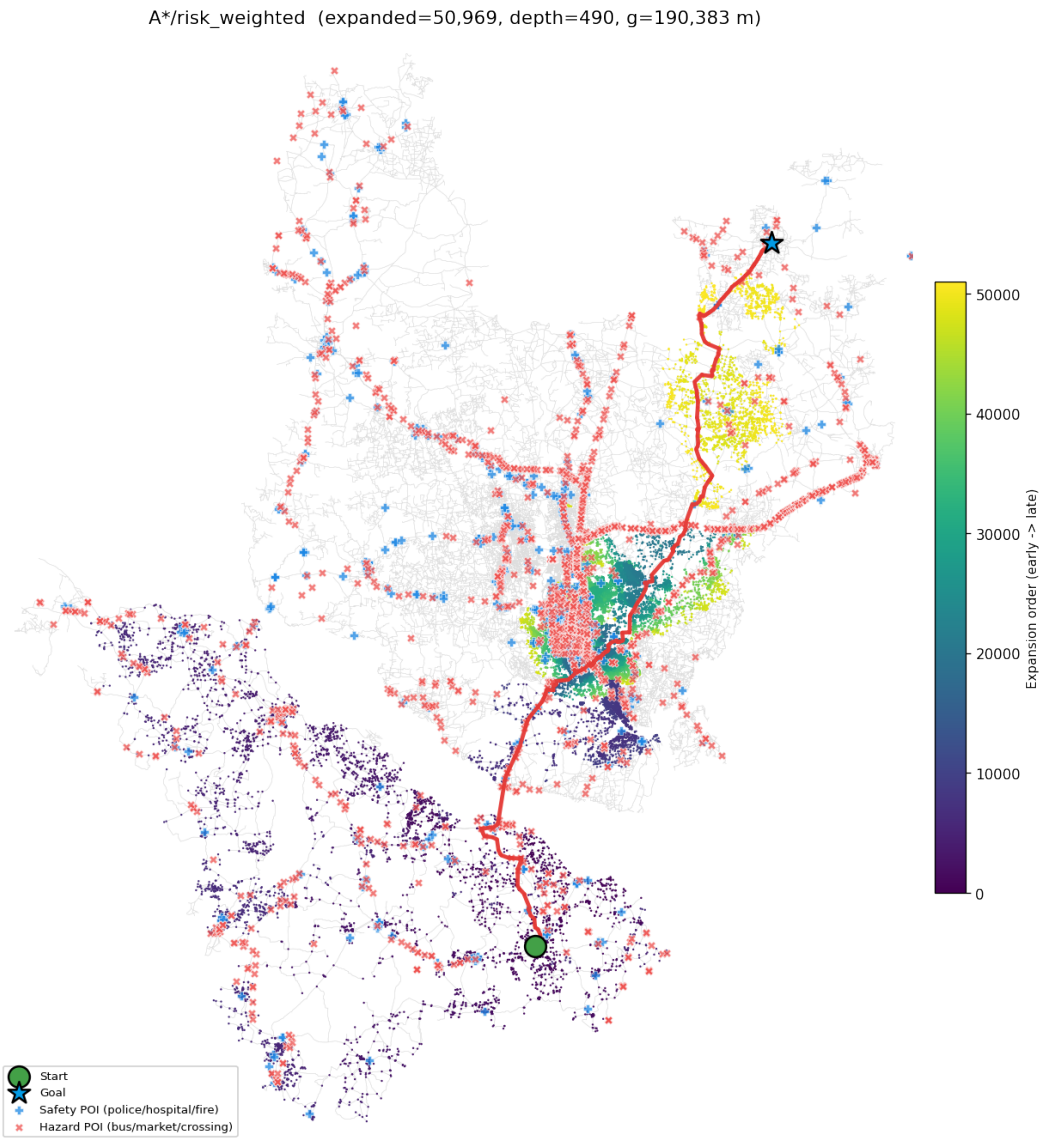


Figure 3: A* with the risk-weighted (inadmissible) heuristic $h_W(n) = h_E(n)(1 + 2\,r(n))$. Fewer expansions but a longer, curvier path that bends around hazard-dense regions.

9.2 Full algorithm comparison

Table 4 reports all seven algorithms on the same start/goal pair. Greedy, A*, and WA* are each run with all three heuristics. IDS is the only algorithm that fails to terminate within the 30 s cap.

Table 4: All algorithms, same Dhaka problem. For rows with a heuristic, the heuristic name is appended to the algorithm name.

Algorithm / heuristic	Depth	G_d (m)	G_a (m)	Expanded	Max front.	Runtime (s)	$b^\star$
BFS	263	232,131	318,088	102,591	1,169	0.20	1.031
DFS	10,928	2,690,529	3,633,277	45,764	7,709	0.11	1.000
UCS	399	173,686	235,615	101,165	483	0.44	1.019
IDS	**NOT FOUND** (expanded = 45,650,247, timeout 30.0 s)						
Greedy / Euclidean	567	245,694	339,917	1,284	472	0.01	1.003
Greedy / Haversine	567	245,694	339,917	1,284	472	0.01	1.003
Greedy / Risk-weighted	973	265,955	346,434	14,110	1,265	0.06	1.004
A* / Euclidean	399	173,686	235,615	67,049	1,300	0.49	1.018
A* / Haversine	399	173,686	235,615	67,236	1,305	0.48	1.018
A* / Risk-weighted	490	190,383	255,562	50,969	3,541	0.29	1.013
WA* / Euclidean	524	187,432	258,760	9,487	752	0.04	1.008
WA* / Haversine	524	187,432	258,760	9,708	782	0.06	1.008
WA* / Risk-weighted	455	201,761	268,836	25,257	2,260	0.13	1.013

9.3 Uninformed algorithm visualisations

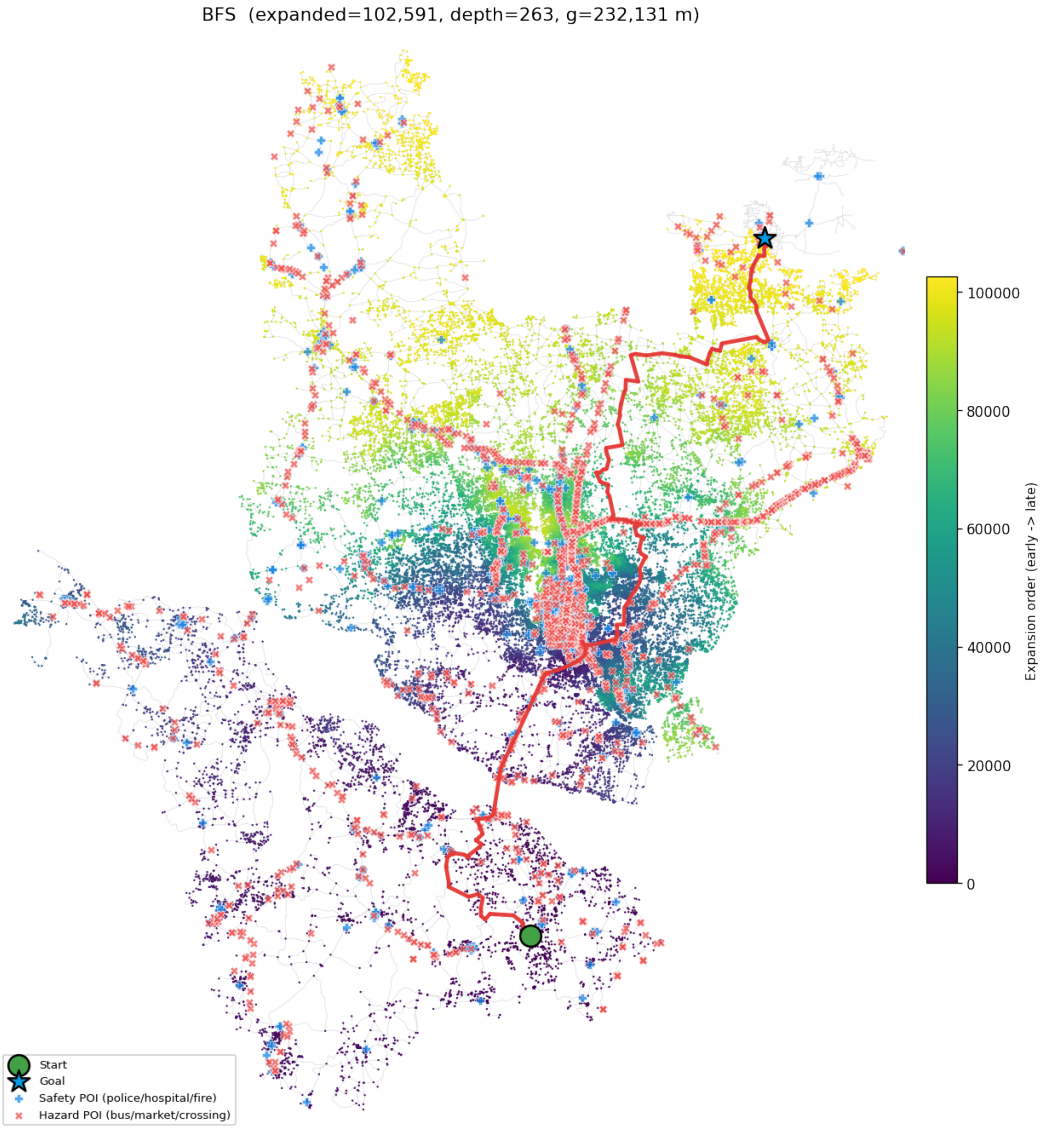


Figure 4: BFS. FIFO-order exploration produces a dense, nearly-circular blast-radius pattern: the search pushes out level-by-level until it reaches the goal, so the expanded region is close to maximal for the given hop-count.

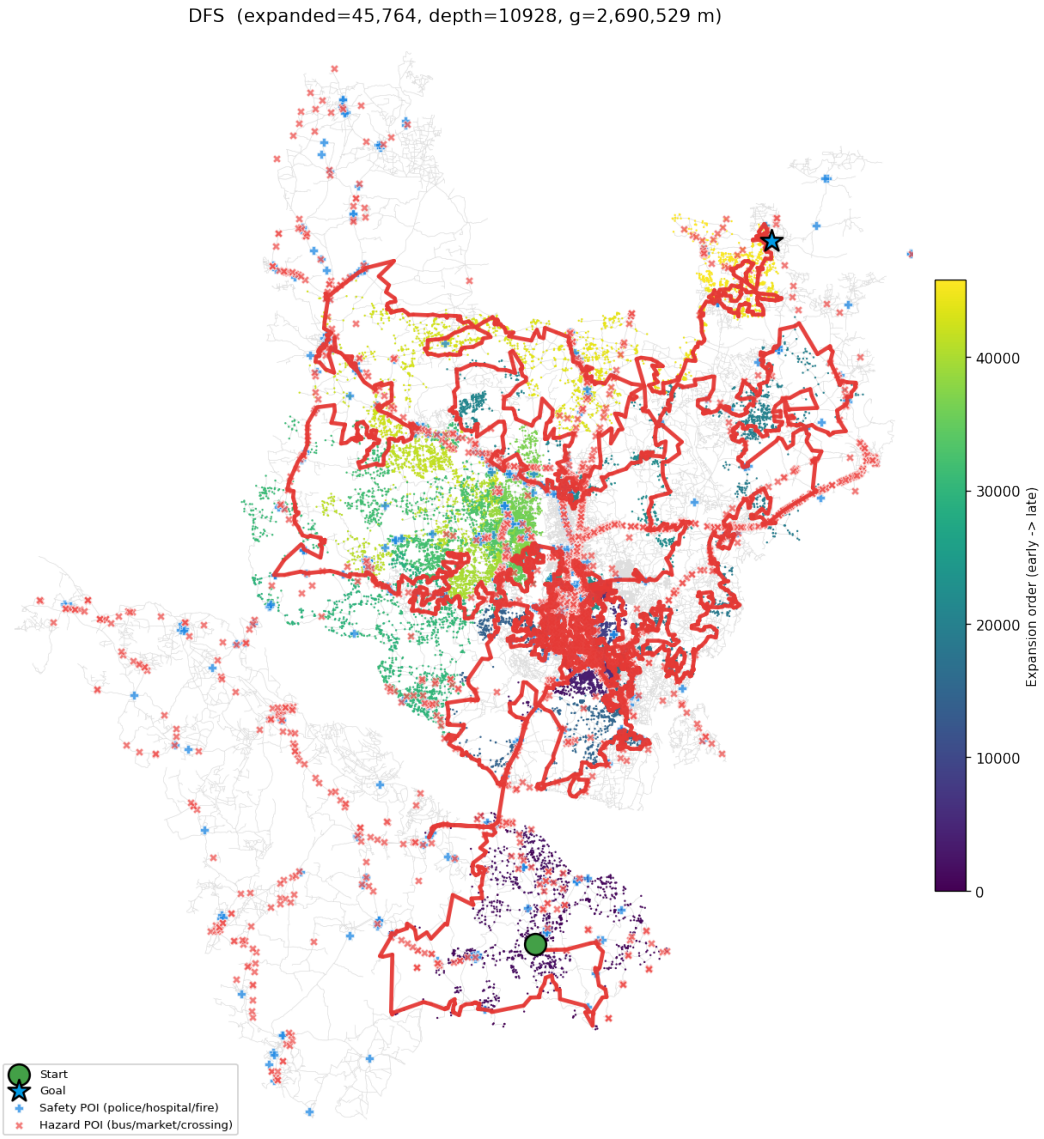


Figure 5: DFS. The red path is the first-found DFS solution and it snakes 10,928 hops across the city because DFS ignores edge length. $G_d \approx 2{,}690{,}529\,\mathrm{m}$: roughly $15\times$ the UCS-optimal path of 173,686 m.

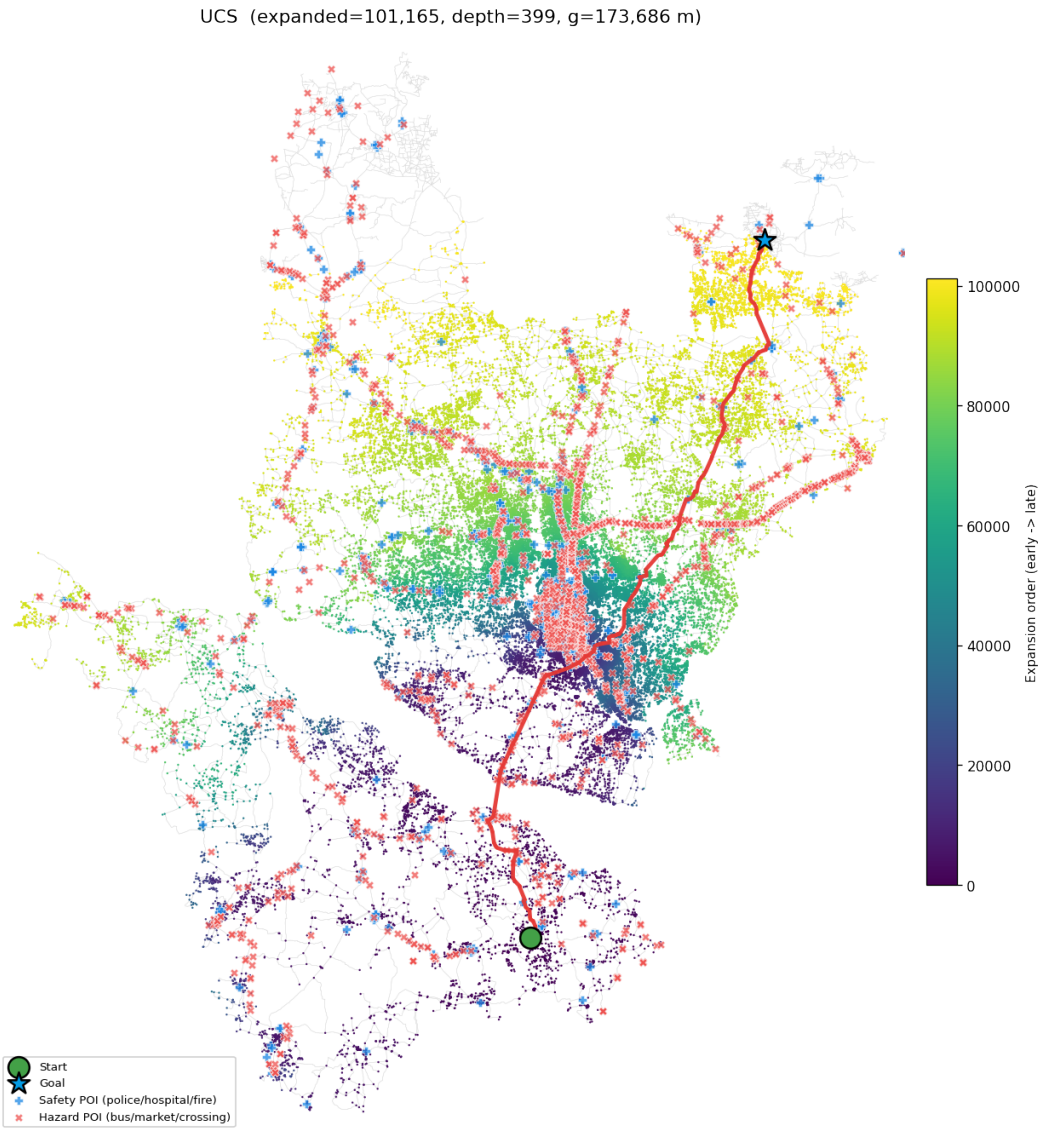


Figure 6: UCS. Cost-priority expansion produces the same optimal path as A* (G_d = 173,686 m, 399 hops), but expands $\approx$1.5$\times$ more nodes because it has no heuristic pull toward the goal.

9.4 Greedy and WA* visualisations

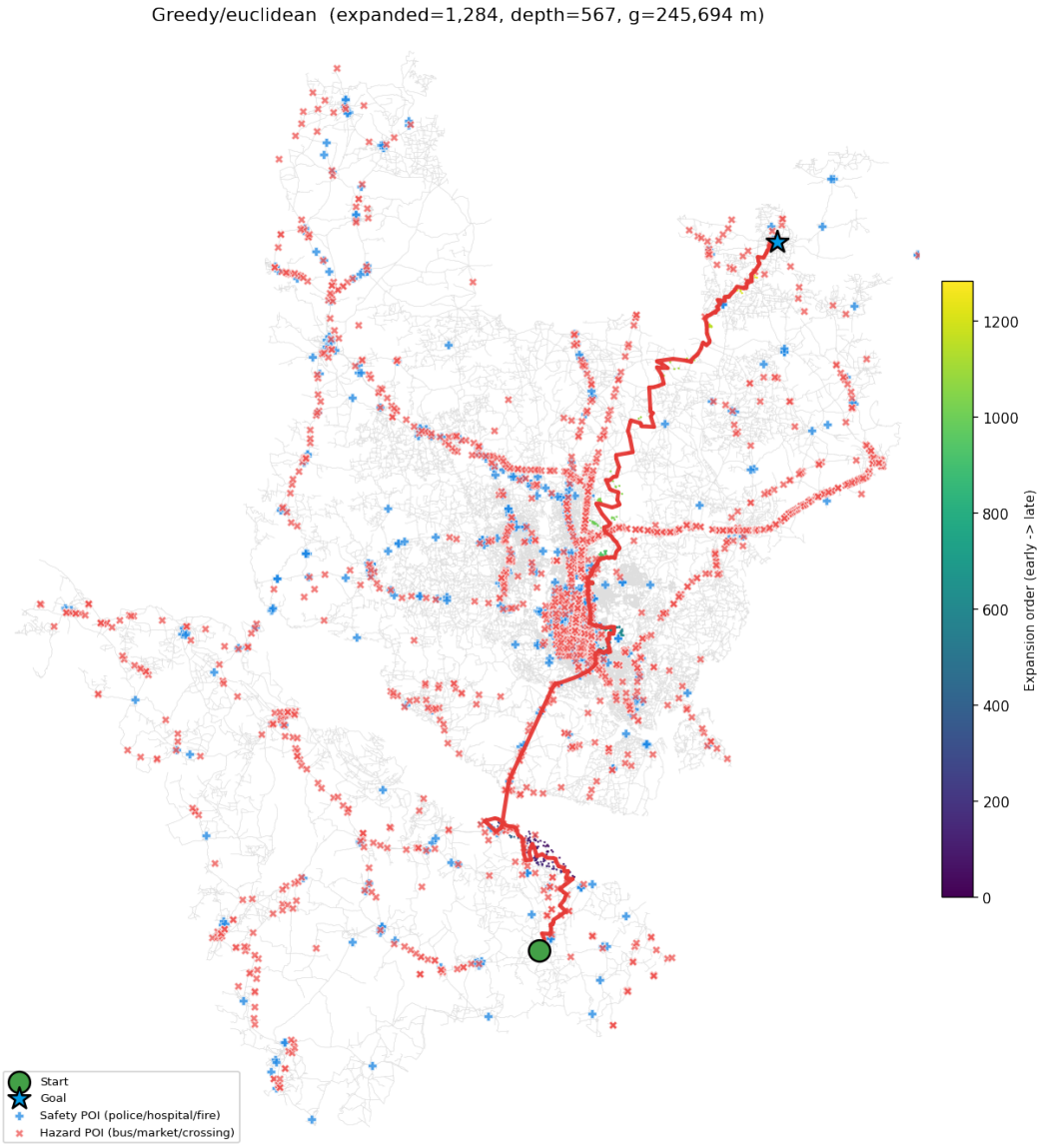


Figure 7: Greedy Best-First with Euclidean heuristic. Only 1,284 expansions — the smallest count in the whole benchmark — but the path is 41% longer than optimal (245,694 m vs. 173,686 m).

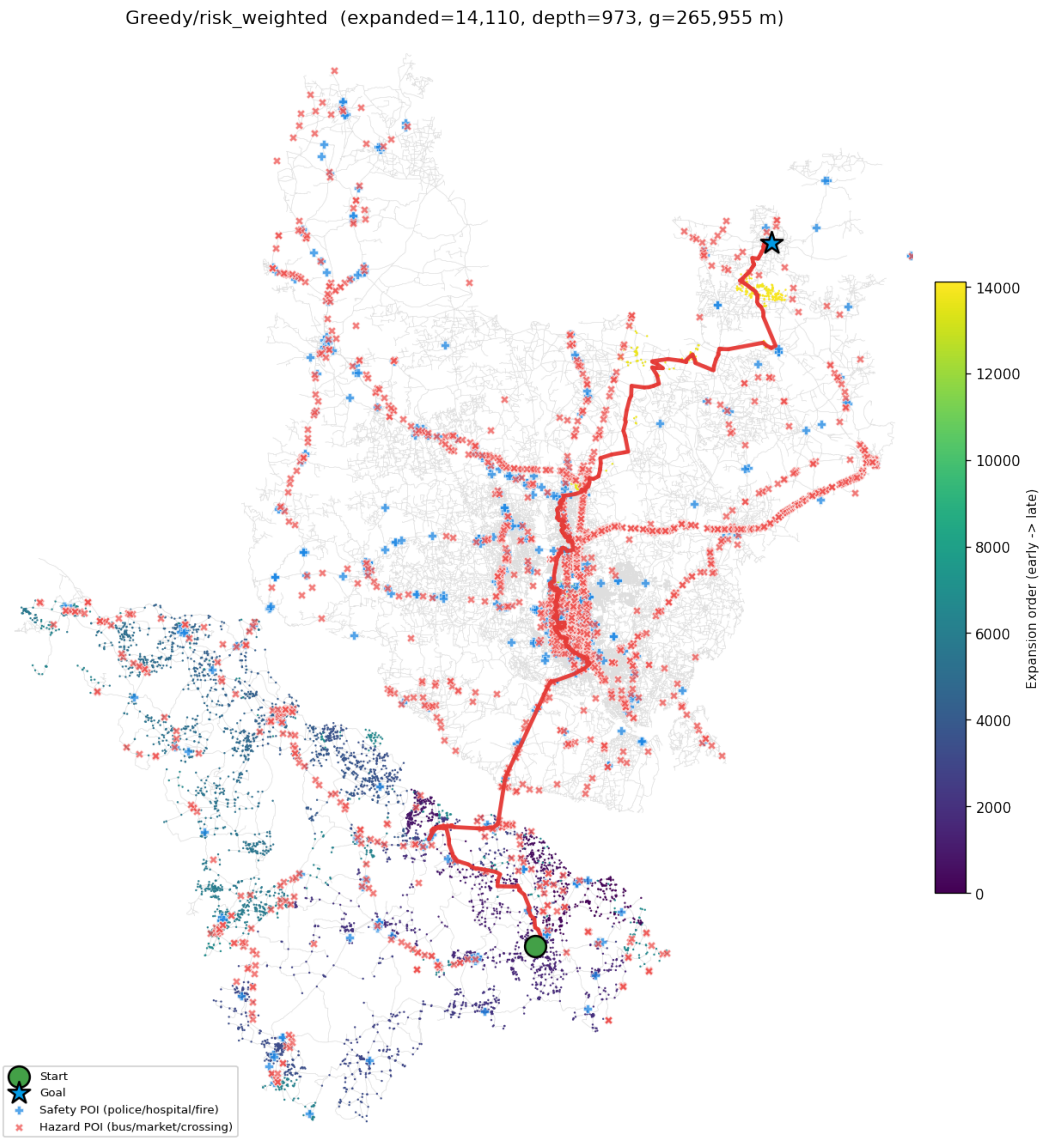


Figure 8: Greedy Best-First with the risk-weighted heuristic. Bending around risk corridors inflates the path to 973 hops and drives 11× more expansions than the Euclidean Greedy.

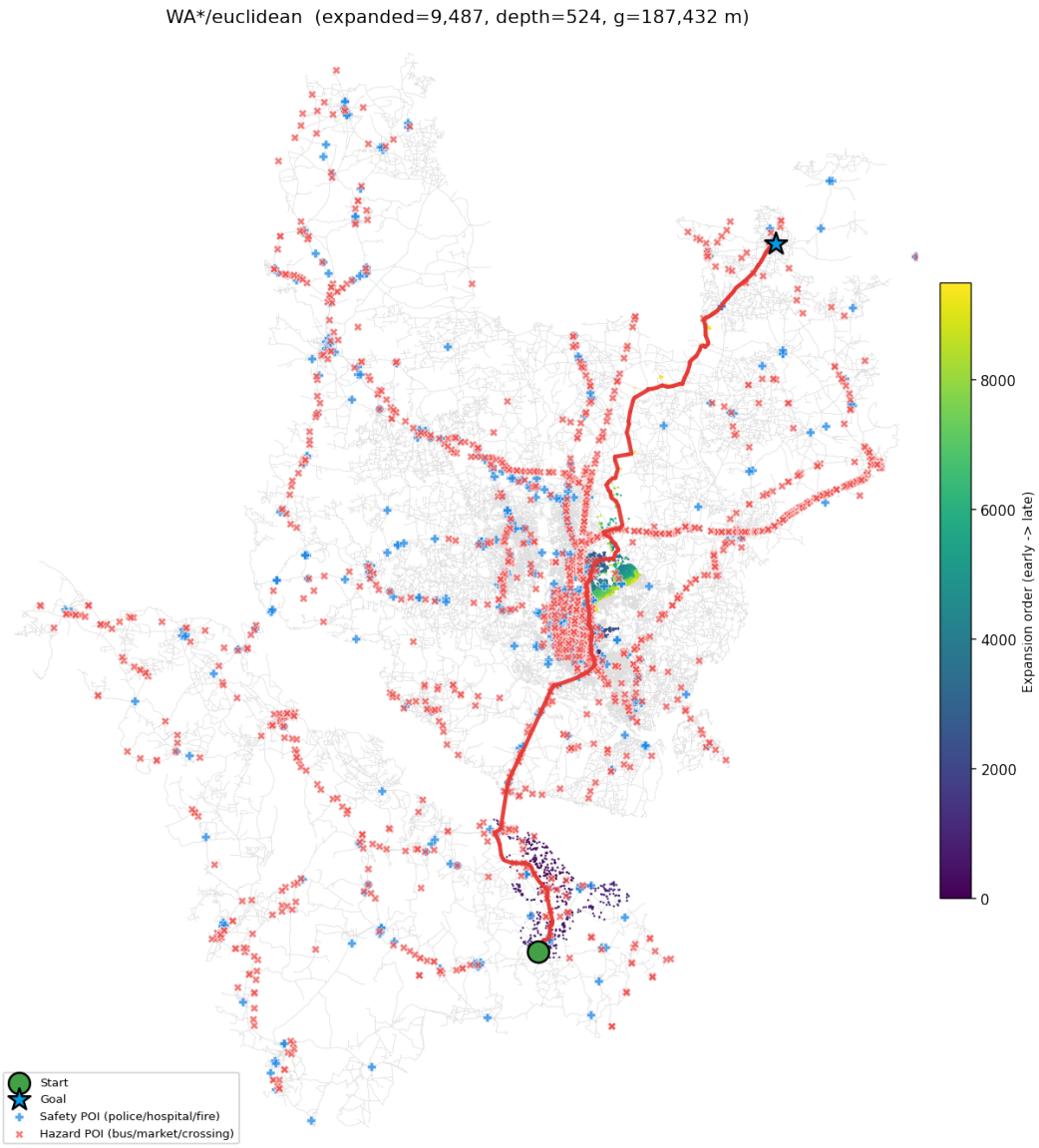


Figure 9: Weighted A* ($w = 1.5$) with Euclidean heuristic. A tight, goal-directed exploration cone: only 9,487 expansions ($\approx 7\times$ fewer than A*) at the cost of an 8% longer path.

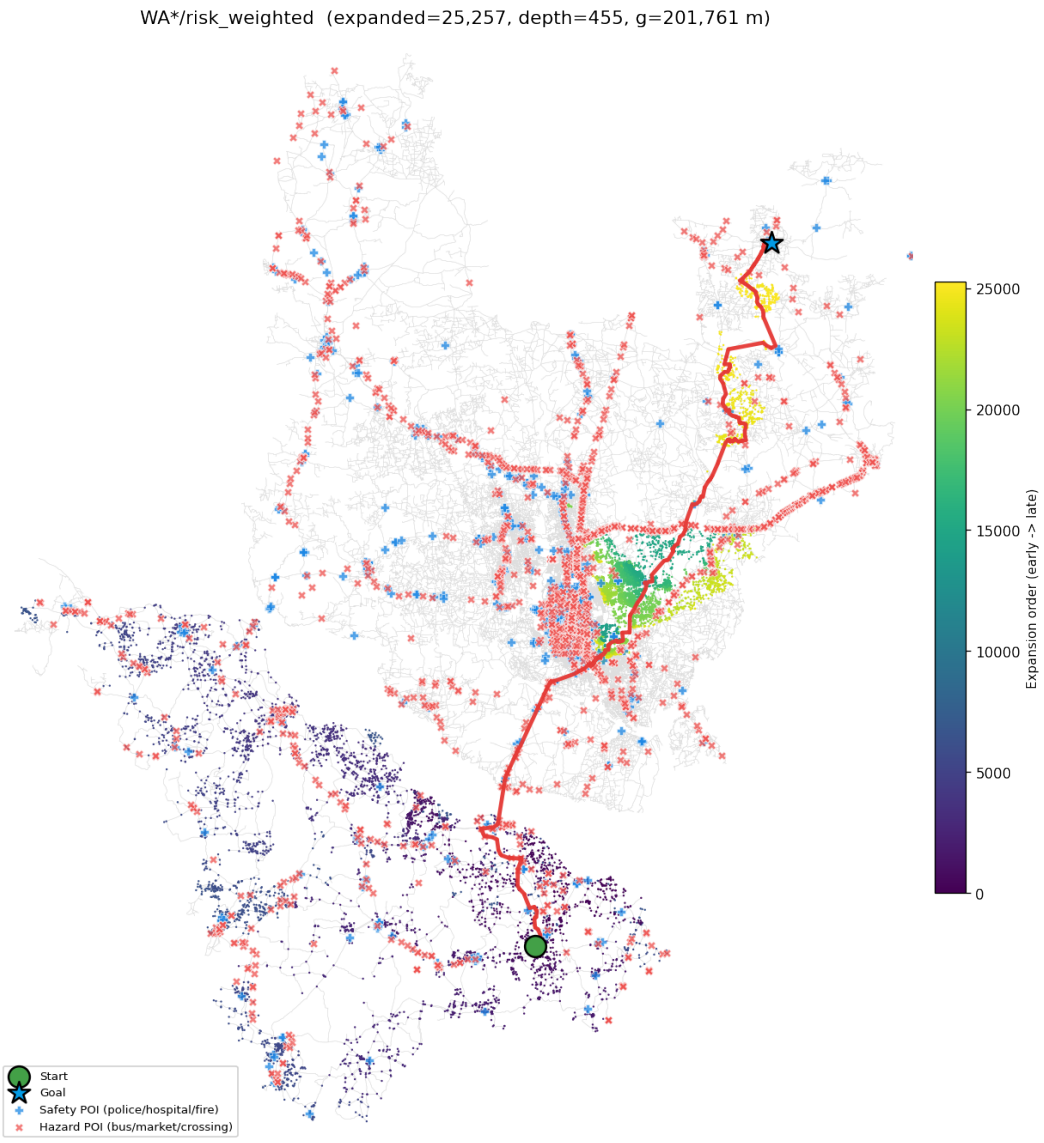


Figure 10: Weighted A* with the risk-weighted heuristic. Combines the goal-pull of weighting with the hazard-avoidance of h_W; expansion is larger than h_E-weighted but still far below standard A*.

9.5 Settings sweep: varying the risk weight α

Table 5 sweeps the simulator's risk weight α while keeping the search (and therefore the path) fixed. Only the *actual* cost changes, because the search is blind to α.

Table 5: Simulator-only sweep over α on the A*/Euclidean path.

α	G_d (m)	G_a (m)
0.5	173,686	198,225
1.5	173,686	235,615
3.0	173,686	291,699

10 Discussion

BFS vs. UCS. BFS returns a *hop-optimal* path of 263 hops, but its $G_d = 232{,}131\,\text{m}$ is 33% longer than the UCS/A* optimum of 173,686 m. This is exactly the textbook warning: the fewest-hops path is not the shortest path when edge lengths vary. BFS also expands more nodes (102,591) than UCS (101,165), a reminder that UCS prunes aggressively using its cost-priority queue while BFS fans out blindly.

DFS. DFS is a clean illustration of *non-optimality*. Its first path is 10,928 hops long, $G_d = 2{,}690{,}529\,\text{m}$ ($\approx 15\times$ the optimum), and its peak frontier of 7,709 nodes is the largest memory footprint in the entire benchmark — the opposite of the textbook "DFS is linear-memory" intuition, because the stack here is dominated by outstanding branch alternatives on a high-degree city graph rather than a single path. The reported $b^\star = 1.000$ is a rounding artefact: with $d = 10{,}928$, solving (11) places $b^\star$ extremely close to 1, i.e. DFS behaves like a very long chain rather than a branching tree.

UCS vs. A*. UCS and A* return the *same* $G_d = 173{,}686\,\text{m}$ at 399 hops — as expected, since h_E and h_H are admissible and consistent and A* then inherits UCS-optimality. A* however expands $\approx 34\%$ fewer nodes (67,049 vs 101,165) and keeps a peak frontier of 1,300 vs. UCS's 483. Note the direction of the trade-off: A* spends *more* peak memory (the f-heap holds many partially-expanded neighbours of the directed front) in exchange for much less total work.

IDS. IDS is the only algorithm that does not finish. By 30 s it has expanded 45,650,247 nodes across $\sim$ 400 depth-limited iterations without reaching the goal. This matches the theoretical prediction exactly: the solution depth is on the order of 400 hops with an effective branching ≈ 2.5, so each deeper iteration costs roughly $2.5\times$ the previous one, and a single iteration at $d \approx 400$ is already well beyond any sensible time budget.

Greedy. Greedy Best-First is the *fastest* search in the benchmark under h_E or h_H: only 1,284 expansions and 10 ms wall-clock. But the price is severe: the path is 567 hops and $G_d = 245{,}694\,\text{m}$, 41% longer than optimum. The simulator amplifies this: $G_a = 339{,}917\,\text{m}$ is 44% worse than the A*/UCS result. This is the classic "hop where it looks closest, ignore how expensive it already was" failure mode.

Greedy with h_W. Replacing h_E with h_W inside Greedy more than halves the frontier quality and costs $11\times$ the expansions (14,110 vs 1,284), because h_W adds a "move away from hazard" pressure that conflicts with the "move towards goal" pressure Greedy relies on. G_d also grows from 245,694 m to 265,955 m. In other words, for Greedy, inadmissibility through h_W is pure downside.

Euclidean vs. Haversine. At Dhaka scale the two admissible heuristics are effectively identical under every informed algorithm. A* expansions differ by only 187 nodes (0.28%); Greedy and WA* return the *same* path with h_E and h_H. This confirms that the Haversine correction is redundant over city-sized regions and that a structurally different heuristic would be needed for a richer three-way comparison.

A* with h_W: the cost of inadmissibility. The risk-weighted heuristic with $w = 2.0$ expands 24% fewer nodes than A$^\star/h_E$ (50,969 vs 67,049), but returns a non-optimal path:

- G_d: 190,383 m vs 173,686 m $\Rightarrow$ **+9.6%**.
- Depth: 490 vs 399 $\Rightarrow$ 91 extra hops.
- G_a (at $\alpha = 1.5$): 255,562 m vs 235,615 m $\Rightarrow$ **+8.5%**.
- Peak frontier: 3,541 vs 1,300 $\Rightarrow$ 2.7$\times$ more memory.

So although h_W is faster per-expansion, it *loses on every metric that matters*: it produces a longer, more memory-hungry, and simulator-worse path. This is a clean demonstration of why admissibility matters when optimality is the goal.

Weighted A* ($w = 1.5$). WA* is arguably the most interesting row in the table. With Euclidean h it needs only 9,487 expansions — 7$\times$ fewer than standard A* and 11$\times$ fewer than UCS — at the cost of an 8% longer path (187,432 m vs 173,686 m) and 24 ms runtime (nearly an order of magnitude faster than A*'s 220 ms). Its peak frontier (752) is also well below A*'s (1,300). In pure "engineering" terms, for Dhaka-scale routing where we can tolerate sub-10% suboptimality, WA* dominates the other informed algorithms.

Why varying α does not change the path. The search objective (4) is distance; α only appears in the simulator (6). The alpha sweep therefore produces *identical* G_d values (173,686 m) and identical expansion counts, while G_a scales linearly with α: 198 km $\rightarrow$ 236 km $\rightarrow$ 292 km for $\alpha = 0.5 \rightarrow 1.5 \rightarrow 3.0$. This cleanly separates "what the planner optimises" from "what the user experiences".

Effective branching factor. All informed runs sit in a narrow band $b^\star \in [1.003, 1.018]$. For a graph with true average out-degree ≈ 2.45, this means the heuristic is pruning the great majority of the search tree: every informed algorithm behaves close to a guided depth-first walk towards the goal. The uninformed algorithms, by contrast, sit at $b^\star = 1.019$ (UCS) and $b^\star = 1.031$ (BFS), the latter being the worst in the benchmark — consistent with its level-by-level blast-radius exploration pattern visible in Figure 4.

Optimality / completeness summary. Matching the results back to the textbook properties:

- **Complete and hop-optimal:** BFS (G_d not optimal), IDS (would be, if it terminated).
- **Complete and cost-optimal:** UCS, A* with admissible h.
- **Complete, not optimal:** DFS (in finite graphs with cycle checks), Greedy, WA*, A$^\star/h_W$.
- **Incomplete in practice under our time budget:** IDS.

The empirical results agree with every one of these statements: UCS and A*/admissible land on the same 173,686 m optimum; BFS hits the fewest-hops solution but not the shortest; DFS, Greedy, WA*, and A$^\star/h_W$ all return sub-optimal paths; IDS times out.

11 Limitations and Future Work

- h_H is effectively identical to h_E at Dhaka scale and should be replaced by a structurally different heuristic (e.g. a landmark / ALT heuristic, or a learned heuristic) to make the three-heuristic comparison informative.
- Endpoints should be averaged over several seeded start/goal pairs so the reported numbers are not $n = 1$. The framework already parameterises the seed, so this is a scripting change rather than a code one.
- Time-of-day factor τ and a road-class modifier were modelled but not exercised in this run; both would be straightforward settings sweeps.
- IDS as implemented is a tree-search IDS without iteration caching. A graph-search variant (e.g. IDA*) would be more competitive and would let us report a non-trivial informed-iterative-deepening comparison against A*.
- w in WA* is fixed at 1.5; a sweep over $w \in \{1.0, 1.5, 2.0, 3.0, \infty\}$ would interpolate smoothly from A* to Greedy and is a natural follow-up.

12 Conclusion

We have built a complete, modular risk-aware search framework on a Dhaka OSM graph and implemented seven search algorithms against the same shared interface. The experimental numbers reproduce every major theoretical property from the lecture: UCS and A* with admissible h land on the G_d optimum; BFS is hop-optimal but distance-suboptimal; DFS returns a path that is 15$\times$ too long; Greedy is the fastest but worst-in-cost informed algorithm; Weighted A* trades near-optimality for an order-of-magnitude speedup; and IDS times out exactly where the branching-factor arithmetic predicted it would. The comparison also isolates the cost of inadmissibility: the risk-weighted heuristic h_W produces longer, more memory-hungry, simulator-worse paths under A*, confirming that trading admissibility for expansion savings is not worthwhile on this problem.